

AnyDB CMS

One CMS for every database

Design Documentation

October 2026

Abstract

AnyDB CMS is a database-agnostic CMS and admin layer. It connects to SQL, document, key-value and realtime databases, including cloud-hosted ones such as Firebase, Supabase, Neon, MongoDB Atlas, AWS, Google Cloud and Azure, and gives every client stack one REST/-GraphQL API and one admin UI.

Status: the core, the memory and PostgreSQL adapters, the gateway, the admin app, drafts, publish and live preview are built and tested locally. The early sections describe the full design, parts of which are still future work; the later sections describe what is built and how to build, run and deploy it. The Roadmap section lists what is done and what is next. “anydbcms” is a working name; check domain, package scope and trademark before committing. Time and file-count figures are planning estimates, not benchmarks.

Contents

1	Overview	5
1.1	Problem	5
1.2	Product	5
1.3	Users	5
1.4	Scope	5
1.5	Principles	6
2	What to Build	6
2.1	Components	6
2.2	Adapter families	7
2.3	Per-adapter files	7
2.4	Dependency rules (enforced in CI)	7
2.5	Definition of done for an adapter	7
3	Language Decision	7
3.1	Recommendation	7
3.2	Why TypeScript first	8

3.3	Where TypeScript is weaker	8
3.4	Options compared	8
3.5	Language freedom for adapters	9
4	Architecture	9
4.1	System context	9
4.2	Layers	9
4.3	Read request flow	9
4.4	Connect-and-introspect flow	10
4.5	Capability handling	10
4.6	Multi-tenant bring-your-own-database	10
4.7	Deployment view	10
5	Adapter Contract	10
5.1	Required operations	11
5.2	Capability flags	11
5.3	Mapping rules per family	11
5.4	Error model	11
5.5	Conformance suite	12
5.6	Adding a new adapter	12
6	Data Model and Schemas	12
6.1	Control-plane entities	12
6.2	Canonical content model	12
6.3	Type mapping (canonical to native)	13
6.4	Portable query AST	13
6.5	Rules	14
6.6	Control-plane DDL	14
6.7	Query AST JSON Schema	16
7	Security	17
7.1	Threat model	18
7.2	Credentials	18
7.3	Least privilege	18
7.4	Network controls	18
7.5	Injection defence	19
7.6	Access control, audit and testing	19
8	API	19

8.1	REST endpoints (v1)	19
8.2	Pagination, errors and versioning	20
9	SDKs	20
9.1	Hand-written layer	21
9.2	Rules	21
10	Deployment	21
10.1	Environments and services	21
10.2	Cloud identity and networking	22
10.3	Configuration, observability, CI/CD	22
11	Gateway	22
11.1	Many databases at once	23
11.2	Security controls	23
11.3	Known limits	23
12	Drafts, Publishing and Live Preview	23
12.1	What the editor sees	24
12.2	Three protections	24
12.3	Storage	24
12.4	Live preview	24
12.5	API summary	25
13	Build and Run	25
13.1	Requirements	25
13.2	Build and start	25
13.3	Connect a database	25
13.4	Tests	26
13.5	Production settings	26
14	CI/CD Pipelines	26
14.1	What runs where	27
14.2	How the linked repositories are built	27
14.3	Release image	27
14.4	Not covered yet	28
14.5	Tooling decision: Node and npnm	28
15	Deploy to Production	28

15.1 What you deploy	28
15.2 Steps	28
15.3 Day-two operations	29
15.4 Limits to plan around	29
15.5 Checklist for an agent doing a deploy	29
16 Logging and Monitoring	29
16.1 What is logged	29
16.2 Metrics	30
16.3 The audit log	30
16.4 Limits	30
17 Moving Data Between Databases	30
17.1 How it works	30
17.2 What the plan blocks	31
17.3 Ids and references	31
17.4 What it does not do	31
17.5 Verified	31
18 Roadmap	32
18.1 Hand-written files for the MVP (estimate)	33
18.2 Risks	33
A Adapter Protocol (proto)	34
B Architecture Decision Records	35

1 Overview

1.1 Problem

Content tools are tied to one database. Strapi and Directus are SQL-only, FireCMS is Firestore-first, and internal-tool builders such as Retool and Appsmith reach many databases but are not content CMSs. Teams running several stacks end up with several admin panels.

1.2 Product

AnyDB CMS is a database-agnostic CMS and admin layer:

- Connect any supported database, including bring-your-own cloud databases.
- Introspect it, describe it as a canonical content model, and edit it through one admin UI.
- Expose one REST/GraphQL API so every client stack uses the same contract.
- Keep credentials server-side, read-only by default, with a full audit trail.

1.3 Users

User	Needs
Developer	Connect a database, get an API and SDK
Editor	Edit content without touching the database
Admin / security	Control who sees what, rotate credentials, read audit logs
Tenant (multi-tenant mode)	Bring their own database safely

1.4 Scope

In scope for the MVP:

- Adapters: Postgres, MySQL, MongoDB, Firestore, SQL Server, DynamoDB.
- Introspection, content-model overlay, CRUD, cursor pagination.
- RBAC with field and row rules, SSO/OIDC.
- Encrypted credential vault, audit log, SSRF guard.
- REST, GraphQL and OpenAPI; a TypeScript SDK and generated Dart and Python SDKs.
- Admin UI.

Later: Cosmos DB, Spanner, BigQuery, Redshift, Redis, Firebase RTDB, realtime subscriptions across all adapters, gRPC, and adapters in other languages through the adapter protocol.

Non-goals:

- Not a database or migration tool. It reads and writes the customer's database; it does not replace it.
- Not a cross-database transaction manager. Cross-source relations are resolved at the gateway, not made atomic.
- Not a BI or analytics tool. Warehouse adapters (BigQuery, Redshift) are read-only by default.

1.5 Principles

1. **Security first.** Credentials never reach a browser or mobile app.
2. **Honest capabilities.** Each adapter declares what it can and cannot do; the UI follows it.
3. **Adapters are isolated.** One slow or hostile database cannot starve the rest.
4. **One contract.** Clients never talk to a database directly.

2 What to Build

2.1 Components

Component	Purpose	MVP
packages/core	Canonical content model, query AST, capability flags, errors	Yes
protocol/	Language-neutral adapter contract (proto and OpenAPI)	Yes
packages/adapter-sdk	Base adapter, registry, conformance test kit	Yes
packages/adapters/*	One adapter per database	6
packages/vault	Envelope encryption, KMS, key rotation, cloud-identity tokens	Yes
packages/auth	OIDC, RBAC/ABAC, field and row rules	Yes
packages/audit	Append-only audit writer with redaction	Yes
packages/network-guard	SSRF blocklist, TLS rules, egress and tunnels	Yes
apps/api	Gateway: REST, GraphQL, OpenAPI	Yes
apps/worker	Isolated adapter runners, introspection jobs, real-time	Yes
apps/admin	Editor and admin UI	Yes
db/control-plane	Migrations for tenants, connections, metadata, permissions, audit	Yes
sdk-ts	Hand-tuned TypeScript client	Yes
sdk/dart, sdk/python	Generated from OpenAPI plus a small hand-written layer	Yes
infra/	Docker, Terraform, Kubernetes	Yes
tests/	Conformance, security, end-to-end	Yes

2.2 Adapter families

Family	Adapters	Notes
SQL	postgres, mysql, mssql	postgres covers Supabase, Neon, RDS, Aurora, Cloud SQL, AlloyDB, Azure PostgreSQL
Document	mongodb, firestore, cosmosdb	mongodb also covers Atlas, DocumentDB (feature gaps), Cosmos Mongo API
Key-value	dynamodb, redis	Key conditions only; scans are explicit and costly
Realtime	firebase-rtdb	Path-based
Analytics	bigquery, redshift	Read-only by default; later
Other cloud	spanner, athena, opensearch	Later, on demand

2.3 Per-adapter files

Every adapter folder has the same layout, so adding a database is a copy-and-fill job:

```
package.json  tsconfig.json  README.md
src/index.ts  adapter.ts      config.schema.ts  connection.ts  auth.ts
src/introspect.ts  capabilities.ts  query-compiler.ts  type-map.ts
src/operations.ts  pagination.ts   realtime.ts        errors.ts
tests/conformance.test.ts  compiler.test.ts  introspect.test.ts  security.test.ts
```

2.4 Dependency rules (enforced in CI)

- `core` imports nothing internal.
- `adapters/*` depend only on `core` and `adapter-sdk`.
- `admin` talks only to `api` or `sdk-ts`, never to a database.
- Only `api` and `worker` may import `vault`.

2.5 Definition of done for an adapter

1. Passes the shared conformance suite.
2. Passes the security suite (injection, identifier allowlist).
3. Declares accurate capability flags.
4. Documents supported services, auth modes and limits in its README.

3 Language Decision

This is a recommendation based on the project's needs, not a measured benchmark.

3.1 Recommendation

Use **TypeScript for the whole MVP**. Add **Go** later for performance-critical or third-party adapters through the adapter protocol. Dart and Python SDKs are generated from OpenAPI.

Part	Language	Why
core, adapter-sdk, api, worker	TypeScript (Node.js)	One language across server and UI, shared types, strong official SDKs for Firestore, MongoDB, DynamoDB, Postgres
Admin UI	TypeScript, React (Refine)	Backend-agnostic data providers, same types as the API
Control-plane migrations	SQL	Plain and reviewable
SDK: TypeScript	TypeScript	Hand-tuned, first class
SDK: Flutter	Dart	Generated plus a small hand-written layer
SDK: Python	Python	Generated plus a small hand-written layer
Third-party or heavy adapters (later)	Go or any language	Separate process through the protocol

3.2 Why TypeScript first

- The query AST, content model and capability flags are shared by the gateway, adapters, SDK and admin UI. One type system avoids drift.
- Database SDK coverage is best on Node for the targets that matter (Firebase Admin, MongoDB, DynamoDB, pg/mysql2, Azure and Google clients).
- It is the fastest path to a working MVP for one or two engineers.

3.3 Where TypeScript is weaker

- CPU-heavy work and very high connection counts are better in Go or Rust.
- A single event loop needs care: run adapters in worker processes, not inside the API process.

3.4 Options compared

Option	Strength	Weakness	Verdict
TypeScript everywhere	Shared types, fastest MVP, best DB SDK coverage	Weaker for CPU-bound work	MVP choice
Go core	Fast, small binaries, great concurrency	Admin UI still needs TypeScript; two type systems	Good for later adapters
Python core	Rich data libraries	Slower, weaker typing for a gateway	SDK and scripting only
Rust core	Performance and safety	Slow to build, thin cloud-SDK coverage	Not worth it for the MVP
Java / Kotlin	Mature DB drivers	Heavier, slower iteration	Only if a client requires it

3.5 Language freedom for adapters

Adapters run as separate processes speaking the protocol in `adapter.proto`. The gateway does not care what language an adapter is written in. First-party adapters ship in TypeScript and run through the same protocol, with an in-process fast path allowed for trusted ones.

4 Architecture

4.1 System context

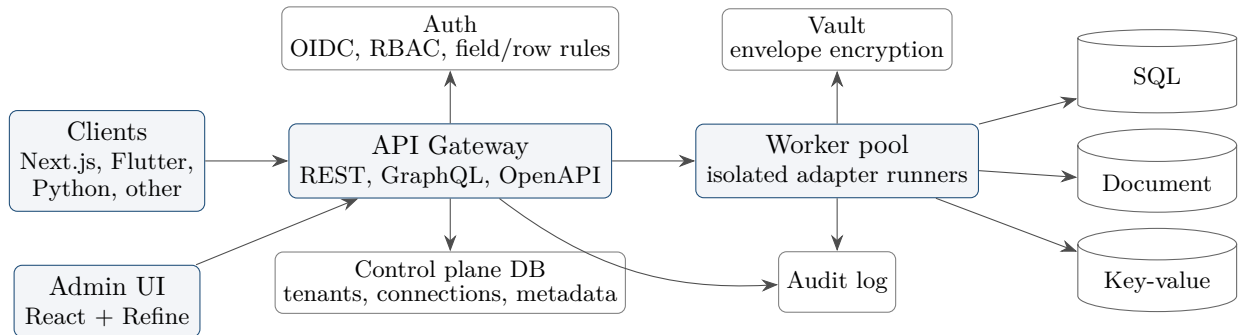


Figure 1: System context. Clients and the admin UI only ever call the gateway.

4.2 Layers

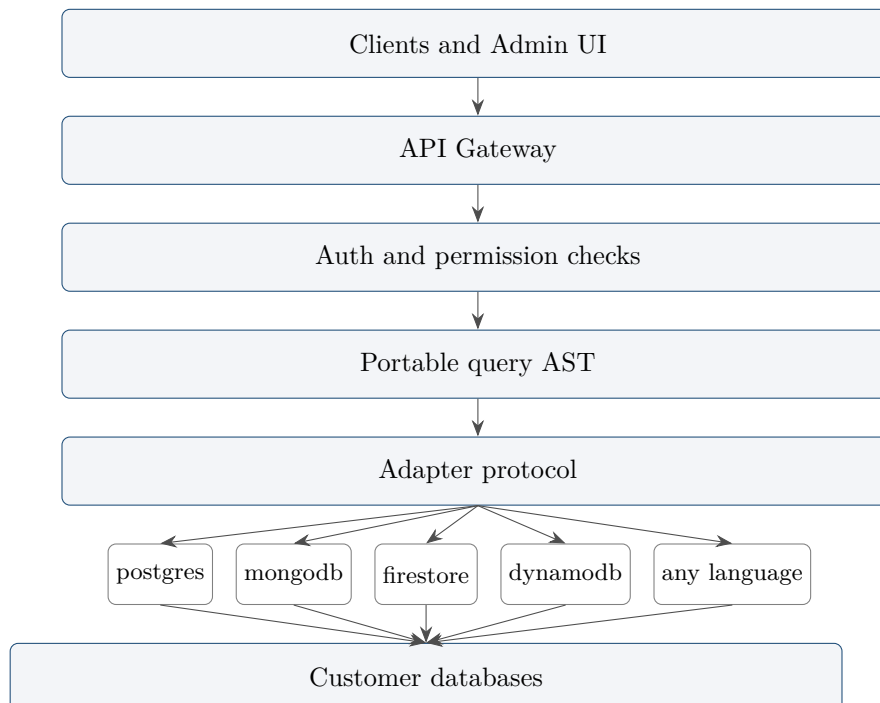


Figure 2: Request path from client to database.

4.3 Read request flow

1. The client calls `GET /v1/connections/{id}/collections/orders/records`.

2. The gateway verifies the token and resolves roles through Auth.
3. The gateway loads connection and collection metadata from the control plane.
4. It builds the query AST and applies field and row rules.
5. It calls `find(ast)` on the worker over the adapter protocol.
6. The worker fetches a short-lived credential or IAM token from the vault.
7. The adapter runs a parameterised native query against the customer database.
8. The worker returns records and a next cursor; the gateway appends an audit event and responds.

4.4 Connect-and-introspect flow

1. An admin posts a new connection (type, host, secret reference).
2. The network guard validates the host (SSRF, TLS, allowlist).
3. The vault encrypts and stores the secret.
4. The control plane saves the connection, read-only by default.
5. A worker reads the catalog or samples documents and saves a draft content model.
6. The admin reviews the draft schema before any collection is exposed.

4.5 Capability handling

When a query uses something an adapter cannot do, the gateway has three outcomes: compile it to the native query (supported), reject it with `CAPABILITY_UNSUPPORTED` (not supported), or emulate it at the gateway and flag it in the response (emulatable). For example, DynamoDB has no arbitrary sort, so sorting by any column is rejected; Firestore has limited OR and inequality filters, so some filters are rejected or emulated and marked as such.

4.6 Multi-tenant bring-your-own-database

The control plane holds tenants, connections, secret references, collection metadata, and roles. The data plane runs one isolated worker per connection (or per tenant). Each connection gets its own pool limits so one tenant's slow query cannot starve another.

4.7 Deployment view

A load balancer with a WAF fronts stateless API pods. API pods use the control-plane Postgres and a job queue. Worker pods read secrets through KMS and reach customer databases through a NAT with static egress IPs, so customers can allowlist the platform. Private networking (PrivateLink, Private Service Connect, Azure Private Link) and SSH tunnels are supported alongside.

5 Adapter Contract

An adapter is the only code that knows a database's native language. It turns the portable query AST into native queries and native results into canonical records.

5.1 Required operations

Operation	Purpose
<code>connect(config)</code>	Open a connection or client using resolved credentials
<code>introspect()</code>	Return collections, fields, types, keys, relations, indexes
<code>capabilities()</code>	Return capability flags
<code>find(query)</code>	List records by filter, sort, projection, cursor
<code>get(collection, id)</code>	Fetch one record
<code>create / update / delete</code>	Write operations (blocked on read-only connections)
<code>batch(ops)</code>	Multiple writes, atomic only if <code>transactions</code> is supported
<code>subscribe(query)</code>	Optional realtime stream
<code>close()</code>	Release resources

5.2 Capability flags

```
interface Capabilities {
  joins: boolean;
  transactions: "none" | "single-document" | "multi-document";
  filters: { or: boolean; inequality: "full" | "single-field" | "none";
    fullText: boolean };
  sort: "any-field" | "indexed-only" | "key-only";
  pagination: { cursor: true; offset: boolean }; // cursor is mandatory
  aggregations: boolean;
  realtime: boolean;
  schema: "enforced" | "inferred";
  writes: boolean;
}
```

The gateway and admin UI use these flags to hide or disable unsupported controls instead of doing silent client-side work.

5.3 Mapping rules per family

Concern	SQL	Document	Key-value
Introspection	Read catalog (<code>information_schema</code>)	Sample documents, infer draft schema	Read key schema, sample items
Filters	Parameterised SQL	Native filter objects	Key conditions only
Pagination	Keyset	startAfter cursor	LastEvaluatedKey
Relations	Foreign keys	Reference fields or embedding	Denormalised
Transactions	Full	Limited	Limited

5.4 Error model

Adapters map native errors to a fixed set so clients see one vocabulary: `CONNECTION_FAILED`, `AUTH_FAILED`, `NOT_FOUND`, `CONFLICT`, `VALIDATION_FAILED`, `CAPABILITY_UNSUPPORTED`, `RATE_LIMITED`, `TIMEOUT`, `READ_ONLY`, `INTERNAL`.

5.5 Conformance suite

A shared test kit runs the same scenarios against every adapter:

1. introspect returns stable collections and field types;
2. create, get, update, delete round trip;
3. cursor pagination returns each record exactly once;
4. unsupported filters return `CAPABILITY_UNSUPPORTED`;
5. a read-only connection rejects writes with `READ_ONLY`;
6. hostile identifiers and operators are rejected (injection);
7. errors map to the standard set.

An adapter is not releasable until all pass.

5.6 Adding a new adapter

Copy the adapter template; fill in the config schema, connection, introspection, query compiler and type map; declare capabilities honestly; register it in the registry; pass the conformance and security suites. Nothing in core, api or admin changes.

6 Data Model and Schemas

Two models exist and must not be mixed. The **control plane** is the platform's own Postgres database (tenants, connections, metadata, permissions, audit). The **canonical content model** is the platform's description of a customer's data; it is stored in the control plane, never inside the customer's database.

6.1 Control-plane entities

6.2 Canonical content model

A **collection** has a name, a native name, fields, relations and capabilities. A **field** has a name, a canonical type, *required* and *unique* flags, validation rules and UI hints. A **relation** has a name, a kind and a mapping to a target collection. Canonical types: string, text, integer, decimal, boolean, datetime, date, json, array, reference, binary, geopoint.

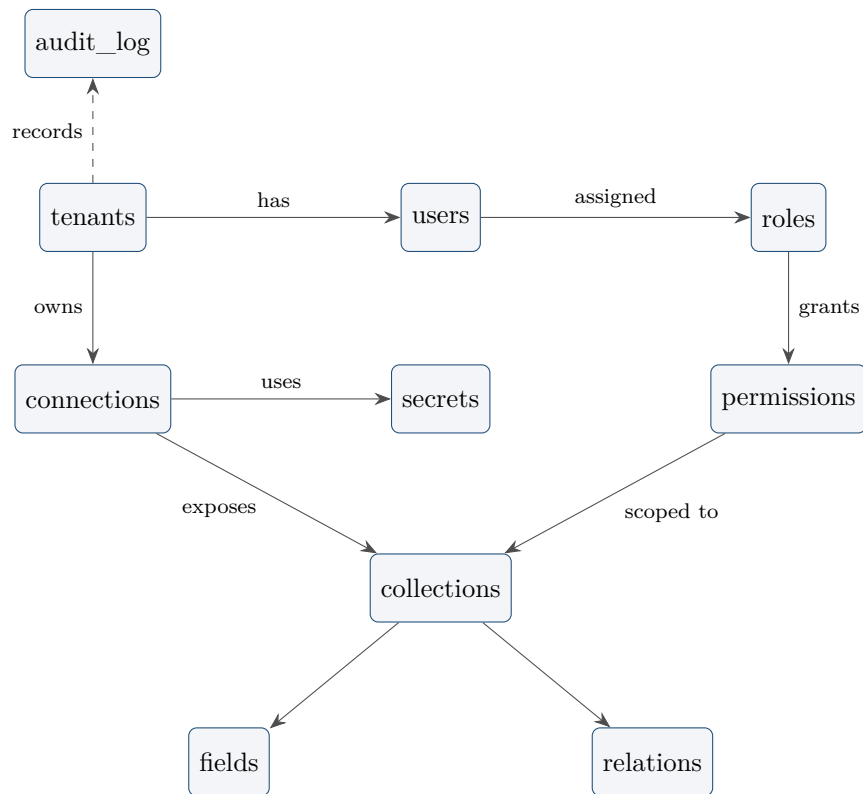


Figure 3: Control-plane entities. The full DDL follows in Section 6.6.

6.3 Type mapping (canonical to native)

Canonical	Postgres	MySQL	MongoDB	Firestore	DynamoDB
string	varchar/text	varchar	string	string	S
integer	int/bigint	int/bigint	int32/int64	integer	N
decimal	numeric	decimal	decimal128	double	N
boolean	boolean	tinyint(1)	bool	boolean	BOOL
datetime	timestamptz	datetime	date	timestamp	S (ISO) or N
json	jsonb	json	object	map	M
array	array or jsonb	json	array	array	L
reference	foreign key	foreign key	ObjectId	document ref	key string
geopoint	point	point	GeoJSON	geopoint	M

Adapters own the exact conversion in `type-map.ts`. Lossy conversions must be flagged during introspection.

6.4 Portable query AST

A query has a collection, an optional filter (and/or/not over comparisons), sort, projection and a page (limit and cursor). Comparison operators: `eq`, `ne`, `lt`, `lte`, `gt`, `gte`, `in`, `nin`, `contains`, `startsWith`, `exists`.

```

{
  "collection": "orders",
  "filter": { "and": [
    { "field": "status", "op": "eq", "value": "paid" },
  ]
}

```

```
{ "field": "total", "op": "gte", "value": 100 }
}],
"sort": [{ "field": "created_at", "dir": "desc" }],
"projection": ["id", "status", "total", "created_at"],
"page": { "limit": 50, "cursor": null }
}
```

Field names are always validated against introspected metadata before compilation; they are never interpolated into native queries.

6.5 Rules

1. Metadata lives in the control plane, never in the customer's database.
2. Cursor pagination is the default everywhere.
3. Cross-source relations resolve at the gateway with batching, not inside any database.
4. Validation happens in the CMS layer for schemaless stores.

6.6 Control-plane DDL

```
-- AnyDB CMS control-plane schema (PostgreSQL)
-- Platform metadata only. Customer data never lives here.

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE tenants (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  email       text NOT NULL,
  oidc_subject text NOT NULL,
  created_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, email),
  UNIQUE (tenant_id, oidc_subject)
);

-- Encrypted connection secrets (envelope encryption; key reference points to KMS).
CREATE TABLE secrets (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  kms_key_ref text NOT NULL,
  ciphertext  bytea NOT NULL,
  auth_kind   text NOT NULL CHECK (auth_kind IN
    ('password', 'service_account', 'iam_token', 'assume_role', 'managed_identity',
    'secret_ref')),
  key_version integer NOT NULL DEFAULT 1,
  created_at  timestampz NOT NULL DEFAULT now(),
  rotated_at  timestampz
);

CREATE TABLE connections (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  name        text NOT NULL,
```

```

adapter_type text NOT NULL,          -- postgres, mysql, mssql, mongodb, firestore,
                                     dynamodb, ...
config        jsonb NOT NULL DEFAULT '{}':jsonb, -- non-secret settings: host, port, db,
                                     tls, pool caps
secret_id     uuid REFERENCES secrets(id),
mode          text NOT NULL DEFAULT 'read_only' CHECK (mode IN ('read_only','read_write')),
status        text NOT NULL DEFAULT 'pending' CHECK (status IN ('pending','active','error','
disabled')),
created_at    timestampz NOT NULL DEFAULT now(),
updated_at    timestampz NOT NULL DEFAULT now(),
UNIQUE (tenant_id, name)
);

CREATE TABLE collections (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  connection_id uuid NOT NULL REFERENCES connections(id) ON DELETE CASCADE,
  native_name   text NOT NULL,
  display_name  text NOT NULL,
  exposed       boolean NOT NULL DEFAULT false, -- nothing exposed until reviewed
  UNIQUE (connection_id, native_name)
);

CREATE TABLE fields (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  collection_id uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
  native_name   text NOT NULL,
  canonical_type text NOT NULL CHECK (canonical_type IN
                                     ('string','text','integer','decimal','boolean','datetime','date',
                                     'json','array','reference','binary','geopoint')),
  required      boolean NOT NULL DEFAULT false,
  is_primary_key boolean NOT NULL DEFAULT false,
  validation    jsonb NOT NULL DEFAULT '{}':jsonb,
  ui            jsonb NOT NULL DEFAULT '{}':jsonb,
  lossy         boolean NOT NULL DEFAULT false, -- native type did not map cleanly
  UNIQUE (collection_id, native_name)
);

CREATE TABLE relations (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  source_collection_id uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
  target_collection_id uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
  name          text NOT NULL,
  kind          text NOT NULL CHECK (kind IN ('one_to_one','one_to_many','many_to_one',
  'many_to_many')),
  mapping       jsonb NOT NULL,          -- which fields join, or reference path
  UNIQUE (source_collection_id, name)
);

CREATE TABLE roles (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  name          text NOT NULL,
  UNIQUE (tenant_id, name)
);

CREATE TABLE user_roles (
  user_id      uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role_id      uuid NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  PRIMARY KEY (user_id, role_id)
);

CREATE TABLE permissions (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  role_id       uuid NOT NULL REFERENCES roles(id) ON DELETE CASCADE,

```

```

    collection_id  uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
    action         text NOT NULL CHECK (action IN ('read','create','update','delete')),
    field_rules    jsonb NOT NULL DEFAULT '{}':jsonb,    -- hide / mask / read-only per field
    row_filter     jsonb,                                -- portable filter AST applied to every
    query
    UNIQUE (role_id, collection_id, action)
);

-- Append-only audit log.
CREATE TABLE audit_log (
    id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tenant_id      uuid NOT NULL REFERENCES tenants(id),
    actor_id       uuid REFERENCES users(id),
    connection_id  uuid,
    collection     text,
    record_id      text,
    operation      text NOT NULL,                      -- read, create, update, delete, connection.create,
    diff           jsonb,                              -- redacted before/after
    ip             inet,
    request_id     text,
    at            timestampz NOT NULL DEFAULT now()
);

CREATE INDEX audit_log_tenant_time ON audit_log (tenant_id, at DESC);
CREATE INDEX connections_tenant ON connections (tenant_id);
CREATE INDEX collections_connection ON collections (connection_id);

-- Make the audit log append-only for the application role.
-- (Run as owner; replace app_role with the real application role.)
-- REVOKE UPDATE, DELETE, TRUNCATE ON audit_log FROM app_role;

-- Optional: row-level security so a request can only see its own tenant.
-- ALTER TABLE connections ENABLE ROW LEVEL SECURITY;
-- CREATE POLICY tenant_isolation ON connections
-- USING (tenant_id = current_setting('app.tenant_id')::uuid);

```

6.7 Query AST JSON Schema

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://anydbcms.dev/schemas/query-ast.schema.json",
  "title": "AnyDB CMS portable query AST",
  "type": "object",
  "required": ["collection"],
  "additionalProperties": false,
  "properties": {
    "collection": { "type": "string", "minLength": 1 },
    "filter": { "$ref": "#/$defs/filter" },
    "sort": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["field"],
        "additionalProperties": false,
        "properties": {
          "field": { "type": "string" },
          "dir": { "enum": ["asc", "desc"], "default": "asc" }
        }
      }
    },
    "projection": { "type": "array", "items": { "type": "string" } },
  },
}

```

```

    "page": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "limit": { "type": "integer", "minimum": 1, "maximum": 500, "default": 50 },
        "cursor": { "type": ["string", "null"] }
      }
    },
    "$defs": {
      "filter": {
        "oneOf": [
          { "$ref": "#/$defs/comparison" },
          {
            "type": "object", "required": ["and"], "additionalProperties": false,
            "properties": { "and": { "type": "array", "minItems": 1, "items": { "$ref": "#/$defs/filter" } } }
          },
          {
            "type": "object", "required": ["or"], "additionalProperties": false,
            "properties": { "or": { "type": "array", "minItems": 1, "items": { "$ref": "#/$defs/filter" } } }
          },
          {
            "type": "object", "required": ["not"], "additionalProperties": false,
            "properties": { "not": { "$ref": "#/$defs/filter" } }
          }
        ]
      },
      "comparison": {
        "type": "object",
        "required": ["field", "op"],
        "additionalProperties": false,
        "properties": {
          "field": { "type": "string" },
          "op": {
            "enum": ["eq", "ne", "lt", "lte", "gt", "gte", "in", "nin", "contains", "startsWith", "exists"]
          },
          "value": {}
        }
      }
    }
  }
}

```

7 Security

Security drives the architecture. This section lists the controls the MVP must have.

7.1 Threat model

Threat	Control
Stolen credentials from the control plane	Envelope encryption; key held in KMS, not in the database
Credentials leaking to clients	Clients only call the API; secrets never leave the vault
SSRF through bring-your-own-database hosts	Block private ranges and metadata endpoints unless explicitly configured
Query injection	Parameterised queries, identifier allowlist from introspected metadata, operator allowlist
Over-broad database access	Dedicated least-privilege role, read-only by default
Cross-tenant impact	Isolated workers, per-connection pool and rate limits
Unaccountable changes	Append-only audit log outside tenant databases

7.2 Credentials

- Encrypt connection secrets with a data key wrapped by a KMS key (envelope encryption), with a documented rotation path.
- Back up the key separately from database backups. Losing it makes secrets unrecoverable.
- Prefer identity-based access over stored passwords: AWS IAM database auth, cross-account roles with external ID and Secrets Manager ARNs; Google service accounts, the Cloud SQL connector and IAM database auth; Azure Entra ID tokens and managed identities (tokens expire, so the vault refreshes them).
- Never log secrets. Redact connection strings in errors.

7.3 Least privilege

- Ask tenants for a dedicated CMS role scoped to the needed schemas or tables.
- Connections are **read-only by default**; write mode is an explicit opt-in.
- **Firebase:** the Admin SDK uses a service account with full access and is not gated by Security Rules. Scope the service account with IAM and enforce authorization in the CMS.
- **Supabase:** the service-role key bypasses row-level security. To enforce RLS, forward the editor's JWT or use a dedicated Postgres role.

7.4 Network controls

- Require TLS (**verify-full** where supported); allow custom CA bundles.
- Publish static egress IPs so tenants can allowlist the platform.
- Support SSH tunnels and private networking.
- Resolve and validate hostnames server-side and re-check after DNS resolution to avoid re-binding.

7.5 Injection defence

- Compile the query AST to parameterised queries only.
- Allowlist table and column names against introspected metadata.
- MongoDB: reject operator injection (`$where`, `$function`, keys starting with `$` from user input).
- Firestore and DynamoDB: validate field paths.
- Raw-query features sit behind a separate permission and are always audited.

7.6 Access control, audit and testing

SSO/OIDC for editors and MFA for admins. RBAC at collection level plus field rules (hide, mask, read-only) and row filters, enforced in the gateway independently of database grants. The audit log is append-only and stored in the control plane; it records actor, tenant, connection, collection, record id, operation, a redacted diff, IP and timestamp, and also logs connection changes, credential reads by workers and permission changes.

The security test suite covers SQL and NoSQL injection, identifier allowlist bypass, SSRF (private IPs, metadata IPs, DNS rebinding), permission bypass, secret leakage in logs and errors, and cross-tenant access. CI generates an SBOM, pins dependencies and scans them on every pull request.

8 API

One contract for every client. REST is the primary surface; GraphQL is generated from the same content model; OpenAPI is the source for SDK generation.

8.1 REST endpoints (v1)

Method	Path	Purpose
POST	<code>/v1/connections</code>	Create a connection (read-only by default)
GET	<code>/v1/connections</code>	List connections
GET	<code>/v1/connections/{id}</code>	Connection detail and health
PATCH	<code>/v1/connections/{id}</code>	Update config, mode, secret reference
DELETE	<code>/v1/connections/{id}</code>	Remove connection and secret
POST	<code>/v1/connections/{id}/introspect</code>	Start an introspection job
GET	<code>/v1/connections/{id}/collections</code>	List collections and fields
PATCH	<code>/v1/collections/{id}</code>	Edit display name, UI hints, exposure
GET	<code>/v1/connections/{id}/collections/{name}/records</code>	List records

Method	Path	Purpose
GET	.../records/{rid}	Get one record
POST	.../records	Create
PATCH	.../records/{rid}	Update
DELETE	.../records/{rid}	Delete
POST	/v1/connections/{id}/batch	Batch writes
GET	/v1/audit	Query the audit log
GET	/v1/health	Liveness and readiness

Write endpoints return `READ_ONLY` on read-only connections.

8.2 Pagination, errors and versioning

Pagination is cursor-only. Cursors are opaque, signed and bound to the query, so a client cannot alter the filter between pages. Offset pagination is offered only where the adapter supports it. Responses carry `capabilityNotes` listing any emulated or degraded behaviour.

Error code	HTTP	Meaning
VALIDATION_FAILED	400	Invalid input
AUTH_FAILED	401	Missing or invalid token
FORBIDDEN, READ_ONLY	403	Not permitted
NOT_FOUND	404	Unknown resource
CONFLICT	409	Write conflict
CAPABILITY_UNSUPPORTED	422	Adapter cannot do this
RATE_LIMITED	429	Too many requests
CONNECTION_FAILED	502	Database unreachable
TIMEOUT	504	Database timed out
INTERNAL	500	Unexpected error

The API is path-versioned (`/v1`); additive changes do not bump the version. Authentication uses a bearer token from the OIDC provider; service tokens are tenant- and role-scoped, rotatable and never accepted from browsers. Writes accept an `Idempotency-Key` header, and rate limits apply per tenant and per connection.

9 SDKs

Every client stack uses the same API. SDKs are generated from `openapi.yaml` so they never drift, with a small hand-written layer for ergonomics.

Language	Package	Approach	
TypeScript / JavaScript	@anydbcms/sdk	Hand-tuned, first class	
Dart / Flutter	anydbcms (pub.dev)	Generated models plus hand-written client	Check
Python	anydbcms (PyPI)	Generated models plus hand-written client	
Kotlin / Java, Swift	generated	Android, JVM, iOS	
Go, C#, PHP, Ruby, Rust	generated	On demand	

package names on npm, pub.dev and PyPI before committing to them.

9.1 Hand-written layer

Each first-party SDK adds the same small set of behaviours: a *client* (base URL, token, request pipeline), *auth* (token refresh), a *query builder* in the language's idiom, *pagination* as an async iterator over cursors, optional *realtime* helpers, and typed *errors* from the API error codes.

9.2 Rules

1. SDKs never hold database credentials. They only hold API tokens.
2. Browser and mobile SDKs use short-lived user tokens, never service tokens.
3. CI regenerates SDKs on every API release and fails if generated code is out of date.
4. Semantic versioning follows the API version.

10 Deployment

Built today: one container that runs the gateway and serves the admin app on a single port, with its state in one volume (see the next sections). This section describes the larger target design: separate api and worker pods, a queue and a managed control-plane database. The worker, the queue and the shared control-plane database are not built yet, and the Kubernetes and Terraform files in `anydbcms-infra` follow the larger design.

10.1 Environments and services

Service	Scaling	Notes	
api	Horizontal, stateless	Never connects to customer databases directly	Environments
worker	Horizontal, per-connection pool caps	Only component that decrypts secrets	
admin	Static assets	Talks only to the API	
control-plane DB	Managed Postgres	Backups, point-in-time recovery	
queue	Managed	Introspection jobs, realtime fan-out	

local (`docker compose` with api, worker, admin, control-plane Postgres and test databases), *staging* (full stack, synthetic tenant data) and *production* (Kubernetes, managed Postgres, KMS-backed secrets).

10.2 Cloud identity and networking

Cloud	Identity (avoid stored passwords)	Private networking
AWS	IAM database auth, cross-account role with external ID, Secrets Manager ARN	PrivateLink, VPC peering
Google Cloud	Service accounts, Cloud SQL connector, IAM database auth	Private Service Connect
Azure	Entra ID tokens, managed identities (refreshed by the vault)	Private Link
Any	Passwords in the vault, SSH tunnels	Static egress IPs

10.3 Configuration, observability, CI/CD

Configuration uses environment variables (names only, never committed values): `DATABASE_URL`, `KMS_KEY_REF`, `OIDC_ISSUER`, `OIDC_CLIENT_ID`, `SESSION_SECRET_REF`, `EGRESS_BLOCKLIST_EXTRA`, `WORKER_MAX_POOL_PER_CONNECTION`, `LOG_LEVEL`. In production, secrets come from a secrets manager, not from `.env` files.

Observability: structured logs with request ids and secrets redacted at the logger; per-connection metrics for latency, errors, pool usage and rate-limit hits; alerts on failed introspection, credential refresh failures and unusual audit patterns.

The CI/CD pipeline runs lint, type-check and unit tests; the conformance suite per adapter against containerised databases; the security suite; image builds with SBOM and dependency scanning; SDK regeneration with drift checks; then deploys to staging, runs end-to-end tests and promotes.

Control-plane backups use automated backups and point-in-time recovery. The KMS key is backed up separately; without it, stored secrets cannot be decrypted even from an intact backup. Run a key-rotation drill before launch.

11 Gateway

The gateway (`anydbcms/apps/api`) is the one process between every client and every database. It holds credentials, applies permissions, keeps drafts and writes the audit log. Clients never talk to a database.

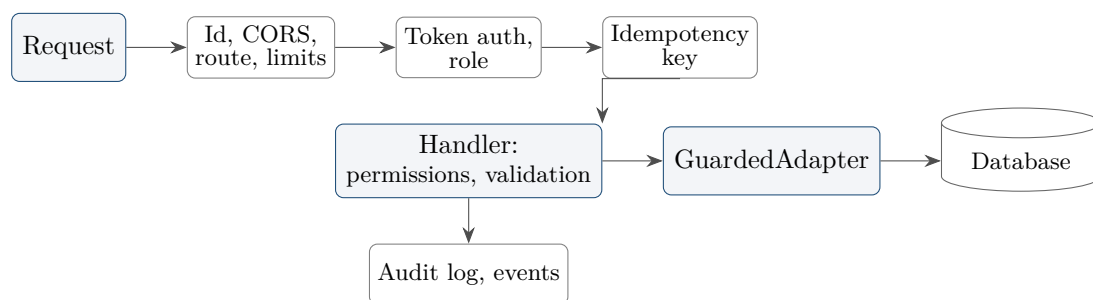


Figure 4: Request pipeline. A failure at any step becomes the standard error body; database text never leaves the server.

11.1 Many databases at once

A connection is a saved row: name, adapter type, mode, settings, encrypted credentials and the collections it exposes. The registry opens an adapter on first use and keeps it open, so memory, PostgreSQL and further connections run side by side, each with its own pool, capabilities and content model. A new connection is read-only and every collection is hidden until an administrator exposes it.

11.2 Security controls

Control	How it works
API tokens	Roles admin, editor, viewer. Only SHA-256 hashes are stored; comparison is timing-safe
Vault	AES-256-GCM, bound to the connection or draft id so ciphertext cannot be moved. Protects database credentials and drafts
Secrets	Vault key, cursor secret and preview secret are required in production; generated into an owner-only file locally
Network guard	Cloud metadata and link-local addresses always blocked; private ranges blocked unless allowed
Rate limits	Per address and per token
Idempotency	Repeating a write with the same Idempotency-Key returns the first answer
Audit	Append-only JSON lines with secrets redacted
Admin page	Served with a strict content security policy

11.3 Known limits

Single process per data folder (file state, per-process rate limits and events); three fixed roles; no single sign-on; events cover only changes made through the gateway; a short DNS-rebinding window in the network guard; adapters run in-process, so gRPC adapters are not wired yet.

12 Drafts, Publishing and Live Preview

Editing never touches the database. Every edit is a draft stored encrypted in the gateway. The database changes only when someone presses Publish, through the same checked write path as any other change.

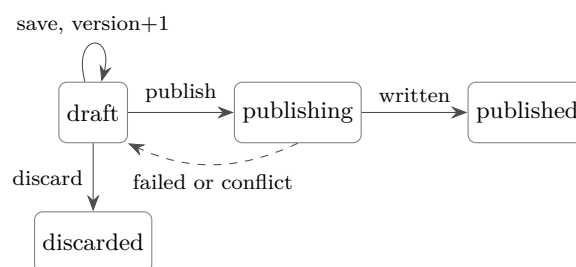


Figure 5: Draft life cycle.

12.1 What the editor sees

Step	Admin app	Gateway
Start typing	“Saving draft”, then “Draft saved”	Creates the draft, version 1
Keep typing	Saves 0.7 s after the last key	Replaces the draft, version + 1
Preview	Pane updates as you type	Streams the draft merged over the published record
Publish	“Published”	Validates, checks for conflicts, writes, removes the draft
Discard	Draft disappears	Marks discarded; database untouched
Record changed elsewhere	Conflict panel	Refuses with 409 <code>record_changed</code>

12.2 Three protections

1. **Version check.** A save states the version it started from; a stale save gets 409 and nothing is lost.
2. **Fingerprint check.** A draft of an existing record stores a hash of that record. If the row changed since (another editor, or a direct database change), publish stops and shows the difference. The editor reloads, or chooses Publish anyway (**force**, recorded in the audit log).
3. **Claim before write.** Publish first moves the draft to **publishing**, so a double click or retry cannot write twice.

12.3 Storage

Drafts live in the gateway’s state store, never in the customer database, encrypted with AES-256-GCM bound to the draft id. Read-only connections can hold drafts, but publish is refused on them. Deleting the gateway data folder deletes unpublished drafts; published content stays in the customer database.

12.4 Live preview

A preview link is `base64url(claims).hmac`. Claims name the draft and the token that created it, and expire after 15 minutes. `GET /v1/preview/:token` returns the published record with the draft applied, masked for the creator; `/events` streams the same payload whenever the draft changes. Revoking the creating token kills the link. A preview link cannot publish or read anything else.

12.5 API summary

Request	Purpose
POST .../collections/:n/drafts	Create a draft
GET .../collections/:n/drafts	List drafts
PATCH .../drafts/:id	Save (send version)
DELETE .../drafts/:id	Discard
POST .../drafts/:id/publish	Publish (version , optional force)
POST .../drafts/:id/preview-link	Create a preview link
GET /v1/preview/:token[/events]	Read, then stream the preview

Only fields in the content model can be saved (unknown fields return 400). Required-field errors appear at publish time so half-written entries can always be saved.

13 Build and Run

13.1 Requirements

Node.js 20 or newer and pnpm, with these repositories cloned side by side: `anydbcms`, `anydbcms-adapter-sdk`, `anydbcms-adapters` and `anydbcms-admin`. Until packages are published, they link to each other by folder, so the folder names matter.

13.2 Build and start

```
cd ~/anydbcms
sh scripts/build-all.sh      # core, adapter-sdk, adapters, gateway, admin
sh scripts/run-local.sh      # http://127.0.0.1:8080
```

The first start prints an administrator token once; paste it into the sign-in page, or set `ANYDBCMS_ADMIN_TOKEN` (32 to 200 characters) beforehand. Data (connections, drafts, tokens, audit log, generated secrets) is kept in `.anydbcms-data/`; back it up and do not commit it.

13.3 Connect a database

Open Connections and add a connection; it starts read-only with all tables hidden. Expose the tables you want, switch to read-write if editing is needed, then edit under Content.

13.4 Tests

Suite	Result recorded
Core	29 tests passing
Gateway	78 tests passing (89 with PostgreSQL), including logging and migration
Live PostgreSQL	Gateway suite passing (injection safety, drafts, direct-change conflict, concurrent databases, read-only); migration suite passing against PostgreSQL 16
Admin app	Type check clean; 15 of 15 browser tests passing

Not yet verified: the build scripts under pnpm on a fresh machine, Docker images, the Dart SDK and the Go adapter server.

13.5 Production settings

Setting	Why
<code>NODE_ENV=production</code>	Refuses to start without secrets; turns off demo data and private networks
<code>ANYDBCMS_VAULT_KEY</code> , <code>ANYDBCMS_CURSOR_SECRET</code> , <code>ANYDBCMS_PREVIEW_SECRET</code>	One random value each; keep in a secrets manager
<code>ANYDBCMS_ADMIN_TOKEN</code>	First administrator; create others, then rotate it
<code>ANYDBCMS_ALLOWED_ORIGINS</code>	Only the sites allowed to call the API
<code>ANYDBCMS_TRUST_PROXY=true</code>	Only behind a proxy you control
<code>TLS</code>	Terminate HTTPS in front of the gateway

14 CI/CD Pipelines

Every repository has its own GitHub Actions workflows. They are kept in `anydbcms/ci-workflows/` and copied into each repository's `.github/` folder with `sh scripts/install-ci.sh`, because tools that edit files for you cannot write into `.github`. Each repository has a “CI OK” job that gathers the results into one check, which is the one to require in branch protection.

14.1 What runs where

Repository	On every push and pull request	On a v* tag
anydbcms	Core and gateway tests on Node 20 and 22, gateway against live PostgreSQL, dependency audit, CodeQL	Builds one container image (gateway with admin inside), pushes it to the GitHub registry, scans it, creates a release
adapter-sdk	Type check, build, tests on Node 20 and 22	Packed package attached to a release
adapters	Same, plus live PostgreSQL	Packed packages attached to a release
admin	Type check, both builds, check that fonts are bundled, CodeQL	Built app attached to a release
sdk-ts	Type check, build, tests on Node 20 and 22	Packed package attached to a release
sdk-python	Tests on Python 3.9 to 3.13, package build	Wheel and source package attached
sdk-dart	Format, analyze, test	Release created
adapter-template-python	Generated protocol code is up to date, tests on Python 3.10 to 3.13	—
adapter-template-go	Generate protocol code, vet, test with the race detector, build	—
infra	Compose, Kubernetes and Terraform checks, configuration scan, Dockerfile lint	—
docs	Link check, schema check, PDF build	PDF attached to a release

Dependabot is switched on in every repository for its package ecosystem and for the workflow actions themselves.

14.2 How the linked repositories are built

Until the packages are published, the repositories link to each other by folder. A workflow therefore checks out its own repository and the ones it needs side by side, then builds them in dependency order: core, adapter SDK, adapters, gateway, admin. If the repositories are private, add a repository secret named `CROSS_REPO_TOKEN` with read access.

14.3 Release image

`anydbcms/docker/Dockerfile.gateway` builds from the folder that holds the four repositories. The image runs as a non-root user, keeps data in a `/data` volume, has a health check, and refuses to start in production mode without the three secrets. The build records provenance and a software bill of materials. The image still contains development dependencies, so it is larger than

it needs to be.

14.4 Not covered yet

Deployment to a cloud (Terraform apply, Kubernetes rollout) needs your cloud credentials and is not written. Nothing is published to npm, PyPI or pub.dev yet. The workflows install with `--no-frozen-lockfile` until lockfiles are refreshed and committed. They have been checked for valid YAML only; they have not run on GitHub.

14.5 Tooling decision: Node and pnpm

The standard is Node 22 with pnpm 9. Bun starts the gateway and passes the unit tests I tried, so it is fine for local use, but it is not the supported path: the folder links, the lockfiles, the workflows and the Docker build were all built and tested on pnpm. Moving to Bun would mean re-testing all of them.

15 Deploy to Production

This is the checklist for running the gateway and admin app for real use. The steps follow the code and the Dockerfile as written. The gateway and its production checks are tested; the container image, the compose and Kubernetes files and the workflows have not been run end to end, so do the verify steps before trusting them.

15.1 What you deploy

One container runs the gateway and serves the built admin app from the same address: one port (8080), one URL, no extra CORS setup. Put a reverse proxy or load balancer in front for HTTPS. A volume mounted at `/data` holds connections, drafts, tokens and the audit log. The gateway reaches your databases from the container.

15.2 Steps

1. **Build the image.** From the folder that holds the four repositories: `docker build -f anydbcms/docker/Dockerfile.gateway -t anydbcms .`, or push a `v*` tag and let GitHub build it.
2. **Create the secrets** once and keep them in a secrets manager: vault key, cursor secret and preview secret (`openssl rand -base64 32` each) and the first admin token (`openssl rand -hex 32`). Back up the vault key separately. Without it, saved database passwords and drafts cannot be decrypted.
3. **Run it** with the volume on `/data`, the four secrets as environment variables, and `ANYDBCMS_TRUST_PROXY=true` only when a proxy you control is in front.
4. **Verify.** `/v1/health` and `/v1/ready` answer; sign in; add a test connection, expose a collection, create a draft, preview it, publish; confirm Audit shows the publish.

In production mode the gateway refuses to start if a secret is missing, turns off the demo connection, and turns off private-network database access unless asked.

15.3 Day-two operations

Task	How
Back up	The <code>/data</code> volume and the secrets; test a restore
Rotate the admin token	Create a new admin token under Access, sign in with it, revoke the old one
Upgrade	Start the new image on the same volume and secrets, run the verify steps, keep the old image for rollback
Logs	One JSON line per request on standard error (see Logging and Monitoring), secrets removed; alert on 5xx and repeated 401 or 429
Database users	Give each connection the fewest rights you can; keep it read-only until editing is needed

15.4 Limits to plan around

One gateway process per data volume (state is a file; rate limits, idempotency keys and live events are per process), so do not run two replicas on the same data. Three fixed roles and no single sign-on. Live events show only changes made through the gateway. The image still contains development dependencies. Deploy workflows (Terraform apply, Kubernetes rollout) are not written.

15.5 Checklist for an agent doing a deploy

Read `AGENTS.md`; confirm the tests pass; build the image and confirm it refuses to start with a secret missing; confirm health and ready; never print or commit secrets; do not run `git commit`, `git push`, `terraform apply` or change a live environment unless the owner said to; report exactly what was verified and what was not.

16 Logging and Monitoring

Everything here is built into the gateway and tested; no extra package is needed. Operational logs go to **standard error** (the start-up banner is the only thing on standard output), as one JSON object per line, or readable text at a terminal. The audit log is a separate file, `audit.log`, in the data folder.

16.1 What is logged

Each request writes one line with a request id (also returned in the `X-Request-Id` header and in every error), a trace id when a W3C `traceparent` header is sent, the method, the path without its query string, the route pattern, the status, the time taken, the caller and role. The caller's IP is added only to 401, 403 and 429 answers. Slow requests (default over one second) and 401, 403 and 429 answers are warnings; 5xx answers are errors; health checks are debug lines. Unexpected failures write an error line with the reason, while the caller only sees a generic `INTERNAL` error. Request bodies, query strings, record values, tokens and passwords are never logged on purpose, and every field also passes a redaction filter.

Setting	Meaning
ANYDBCMS_LOG_LEVEL	debug, info, warn or error (default info)
ANYDBCMS_LOG_FORMAT	json or pretty (default pretty at a terminal outside production, else json)
ANYDBCMS_SLOW_REQUEST_MS	Requests slower than this are warnings (default 1000)
ANYDBCMS_AUDIT_MAX_MB, ANYDBCMS_AUDIT_KEEP	Rotate the audit log at this size; rotated files kept (defaults 10 and 5)
ANYDBCMS_METRICS_TOKEN	Lets a scraper read metrics without an admin token

16.2 Metrics

GET `/v1/metrics` answers in the Prometheus text format and needs an admin token or the metrics token. It reports requests by method, route and status, request duration, in-flight requests, errors by code, rate-limited requests, change events, open event streams, connections by adapter, active tokens, audit entries written, migration records by result, event-loop delay, memory, CPU and uptime. Labels are bounded: routes are patterns, never raw paths, and each metric stops adding label combinations after a fixed limit. Worth alerting on: a 5xx rate above two percent, a 95th percentile above one second, repeated 401 or 429, any failed migration records, and up equal to zero.

16.3 The audit log

Every change to connections, content, tokens and drafts is appended to `audit.log`. Each entry carries the hash of the previous entry, so editing, removing or reordering a line breaks the chain. GET `/v1/admin/audit/verify` checks it and reports where it first breaks. Tampering is shown, not prevented: someone with write access to the volume can rebuild the chain, so keep a copy somewhere the gateway cannot write. The log rotates by size and the chain continues across files.

16.4 Limits

Logs and metrics are per process. The ready check confirms the process is up and does not test each database. Tracing is correlation only: the trace id is copied into logs and no spans are exported. The alert rules are examples and were not tested against a live Prometheus.

17 Moving Data Between Databases

An administrator can copy the records of one connection into another: PostgreSQL to a new PostgreSQL, a demo into a real database, or between any two adapters the gateway supports. It is on the *Move data* page of the admin app and under `/v1/admin/migrations` in the API.

17.1 How it works

1. **Plan** (dry run). The gateway reads both databases' structure and decides how each collection maps. Nothing is written. The plan lists blocking problems, warnings, record counts and how ids are handled.

2. **Run.** The copy runs in the background, one migration at a time. Collections that others point at are copied first. Each record is checked against the target's own rules, then written. Progress can be followed and the run cancelled.
3. **Check.** Record counts are compared before and after, and each stored record is compared with what was sent.

The source is only read, the target must be read-write, and only administrators can start a migration. Record values are never logged, audited or put in error messages. By default records that already exist are left alone, so a stopped or failed run is continued by running it again; **overwrite** replaces them and **fail** reports them.

17.2 What the plan blocks

The run is refused when the target has no matching collection or field, when a source field has nowhere to go (unless **dropUnmappedFields** is set), when a required target field is not filled, when types cannot hold each other's values, or when a collection points at itself while the target rennumbers records. Compatible but different types (string and text, integer into decimal) are warnings, and each value is checked.

17.3 Ids and references

If the target key is an ordinary key, ids are copied as they are. If the target numbers its own records (a PostgreSQL serial column), the target assigns new ids by default and every reference in later collections is rewritten to match; the target collection must then be empty, because running again would duplicate everything. With **preserveIds** the original ids are written into the generated key. PostgreSQL accepts this, after which its sequence must be moved past the largest id; a database that ignores the id makes the run fail with a clear message. References are found from a **references** marker on a field or from a many-to-one relation to the other collection's key, which is how PostgreSQL foreign keys are reported.

17.4 What it does not do

It copies records only. It does not create tables or collections in the target, copy indexes, constraints, views or users, run in one transaction (a failure leaves a partial copy that repeating completes), or freeze the source while it runs. Job progress is kept in memory and is lost if the gateway restarts. For a live system: back up, run once, stop writes, run again.

17.5 Verified

Memory to memory; PostgreSQL to PostgreSQL with ids kept, and renumbered with foreign keys rewritten; memory into PostgreSQL; and PostgreSQL into memory with ids kept, which fails clearly. The PostgreSQL runs used PostgreSQL 16. Not tested: other PostgreSQL versions, very large tables, a live workload. Only the memory and PostgreSQL adapters exist, so migration works between those two for now.

18 Roadmap

Where we are (October 2026)

Phase	State
Foundation	Done: core (29 tests), adapter contract and test kit
Security spine	Mostly done: vault, token auth, audit log, network guard, rate limits, idempotency. Single sign-on not built
First path	Done: PostgreSQL adapter and gateway, tested against a live database
More adapters	Memory and PostgreSQL only; MongoDB, Firestore, MySQL, SQL Server and DynamoDB not started
Admin UI	Done for the first release: connections, content editing with drafts, publish, live preview, access, audit, settings
SDKs	TypeScript, Python and Dart written; Dart and the Go adapter template have not been run
Hardening	Not started: load tests, penetration review, key-rotation drill, cloud-identity connections

Also done beyond the original plan: CI/CD workflows for every repository, a one-image release build, and agent guides. Next, in order: run the full build on a clean machine and fix what it finds; align `anydbcms-infra` with the single-image gateway; add MongoDB, MySQL and Firestore adapters; add single sign-on; add a shared state store so more than one gateway can run.

Estimates assume one or two engineers and are rough planning guesses, not benchmarks.

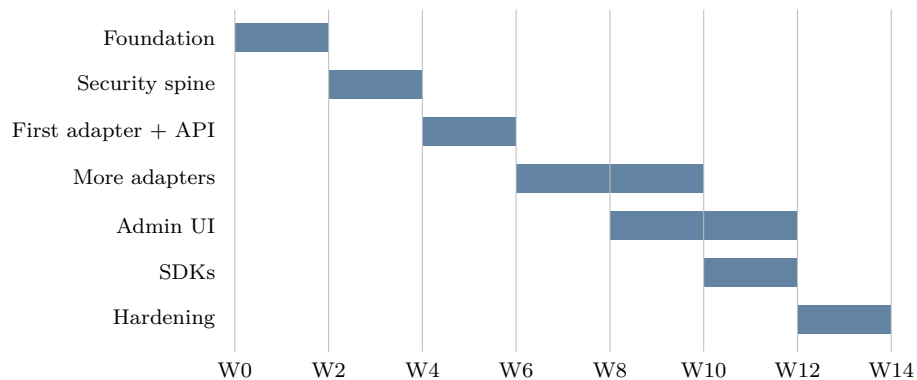


Figure 6: MVP plan in weeks.

Phase	Goal	Exit criteria
1. Foundation	Repo, core types, adapter protocol, control-plane schema	core builds; migrations apply cleanly
2. Security spine	Vault, auth, audit, network guard	Secrets encrypted end to end; SSRF tests pass
3. First path	Postgres adapter plus API	Conformance suite green on Postgres
4. More adapters	MongoDB, Firestore, MySQL, SQL Server, DynamoDB	Each passes conformance and security suites
5. Admin UI	Connect, introspect, edit, permissions	An editor completes a full content edit through the UI only
6. SDKs	Generated Dart and Python, hand-tuned TypeScript	Sample apps (Next.js, Flutter, FastAPI) run
7. Hardening	Load, security review, docs, cloud identity	Security review and key-rotation drill done

18.1 Hand-written files for the MVP (estimate)

Area	Files	Area	Files
Root config	12	api	25
core	15	worker	12
protocol + adapter-sdk	13	admin	40
6 adapters	36	control-plane DB	10
vault	8	sdk-ts	8
auth	10	Dart SDK	8
audit + network-guard	11	Python SDK	9
infra	15	adapter templates (Go, Python)	16
tests	20	CI + docs	20
Total (approximate)			288

18.2 Risks

Risk	Mitigation
Capability gaps make some databases feel second class	Honest capability flags; clear UI messaging
Credential handling mistakes	Security spine before any adapter; key-rotation drill
Scope creep across databases	Ship six adapters; add others on demand
Driver or ORM version changes (for example Prisma and MongoDB)	No single ORM for all stores; isolate drivers inside adapters
Licensing of embedded tools	Prefer MIT/Apache components; legal review before embedding source-available tools

A Adapter Protocol (proto)

```
// AnyDB CMS adapter protocol.
// An adapter implements this service in any language. The gateway calls it.
syntax = "proto3";

package anydbcms.adapter.v1;

import "google/protobuf/struct.proto";

service Adapter {
  rpc Connect(ConnectRequest) returns (ConnectResponse);
  rpc Introspect(IntrospectRequest) returns (IntrospectResponse);
  rpc Capabilities(CapabilitiesRequest) returns (CapabilitiesResponse);
  rpc Find(FindRequest) returns (FindResponse);
  rpc Get(GetRequest) returns (GetResponse);
  rpc Create(CreateRequest) returns (CreateResponse);
  rpc Update(UpdateRequest) returns (UpdateResponse);
  rpc Delete(DeleteRequest) returns (DeleteResponse);
  rpc Batch(BatchRequest) returns (BatchResponse);
  rpc Subscribe(FindRequest) returns (stream ChangeEvent); // optional
  rpc Close(CloseRequest) returns (CloseResponse);
}

// Credentials are resolved by the worker and passed in memory only; never logged.
message ConnectRequest {
  string connection_id = 1;
  string adapter_type = 2;
  google.protobuf.Struct config = 3;
  google.protobuf.Struct credentials = 4;
  bool read_only = 5;
}

message ConnectResponse { bool ok = 1; Error error = 2; }

message IntrospectRequest { string connection_id = 1; }
message IntrospectResponse {
  repeated CollectionInfo collections = 1;
  Error error = 2;
}

message CollectionInfo {
  string native_name = 1;
  repeated FieldInfo fields = 2;
  repeated RelationInfo relations = 3;
}

message FieldInfo {
  string native_name = 1;
  string native_type = 2;
  string canonical_type = 3;
  bool required = 4;
  bool is_primary_key = 5;
  bool lossy = 6;
}

message RelationInfo {
  string name = 1;
  string target = 2;
  string kind = 3;
  google.protobuf.Struct mapping = 4;
}

message CapabilitiesRequest { string connection_id = 1; }
message CapabilitiesResponse {
  bool joins = 1;
  string transactions = 2; // none | single-document | multi-document
  bool filter_or = 3;
}
```

```

string filter_inequality = 4; // full | single-field | none
bool full_text = 5;
string sort = 6;             // any-field | indexed-only | key-only
bool offset_pagination = 7;  // cursor pagination is mandatory
bool aggregations = 8;
bool realtime = 9;
string schema = 10;          // enforced | inferred
bool writes = 11;
}

message FindRequest {
  string connection_id = 1;
  google.protobuf.Struct query = 2; // portable query AST (see query-ast.schema.json)
}
message FindResponse {
  repeated google.protobuf.Struct records = 1;
  string next_cursor = 2;
  repeated string capability_notes = 3;
  Error error = 4;
}

message GetRequest { string connection_id = 1; string collection = 2; string id = 3; }
message GetResponse { google.protobuf.Struct record = 1; Error error = 2; }

message WriteRequest {
  string connection_id = 1;
  string collection = 2;
  string id = 3; // empty on create
  google.protobuf.Struct values = 4;
}
message WriteResponse { google.protobuf.Struct record = 1; Error error = 2; }

message DeleteRequest { string connection_id = 1; string collection = 2; string id = 3; }
message DeleteResponse { bool ok = 1; Error error = 2; }

message BatchRequest { string connection_id = 1; repeated WriteRequest writes = 2; bool atomic = 3; }
message BatchResponse { repeated WriteResponse results = 1; Error error = 2; }

message ChangeEvent {
  string type = 1; // insert | update | delete
  google.protobuf.Struct record = 2;
}

message CloseRequest { string connection_id = 1; }
message CloseResponse { bool ok = 1; }

message Error {
  string code = 1; // CONNECTION_FAILED, AUTH_FAILED, NOT_FOUND, CONFLICT, VALIDATION_FAILED,
                  // CAPABILITY_UNSUPPORTED, RATE_LIMITED, TIMEOUT, READ_ONLY, INTERNAL
  string message = 2;
  google.protobuf.Struct details = 3;
}

```

B Architecture Decision Records

ADR 0001: Language choice

Proposed, 2026-10-08

Decision: TypeScript for core, adapter-sdk, first-party adapters, api, worker and admin; generated Dart and Python SDKs; adapters in any language through a language-neutral protocol. **Revisit when** worker CPU or connection counts become the bottleneck, or a customer needs an adapter

whose best driver is not on Node.

ADR 0002: Adapters as processes behind a protocol

Proposed, 2026-10-08

Decision: define the adapter contract in `adapter.proto` and run adapters as separate worker processes, with an in-process fast path for trusted first-party adapters. **Gains:** any language, per-connection isolation, one conformance kit. **Costs:** a network hop and more services to operate.

ADR 0003: CMS metadata lives in the control plane

Proposed, 2026-10-08

Decision: store all CMS metadata in a separate control-plane Postgres; customer databases hold only customer data; connections are read-only by default. **Cost:** schema drift must be caught by re-introspection and reconciled with stored metadata.